Redes Neurais Convolucionais

Uma leitura do capítulo 9 de *Deep Learning*

Eraldo Silva Junior

29 de outubro de 2026

CPE727 — Aprendizado Profundo
PEE/COPPE/UFRJ

Roteiro

1. A operação de convolução (cap. 9.1)
2. Motivação (cap. 9.2)
3. Pooling (cap. 9.3)
4. Convolução e pooling como um prior infinitamente forte (cap. 9.4)
5. Variantes da função de convolução básica (cap. 9.5)
6. Saídas estruturadas (cap. 9.6)
7. Tipos de dados (cap. 9.7)
8. Algoritmos eficientes de convolução (cap. 9.8)
9. Características (*features*) aleatórias ou não supervisionadas (cap. 9.9)
10. A base neurocientífica das redes convolucionais (cap. 9.10)
11. Redes convolucionais e a história do aprendizado profundo (cap. 9.11)

1. A operação de convolução (cap. 9.1)

1.1 De onde vem a palavra

- Do latim *convolvere*: *con-* (“junto”) + *volvere* (“rolar, girar”).
- *Convolvo*: “enrolar junto, enrolar em torno de si”; daí *convolutio*, “ato de enrolar”.
- Leitura do nome (interpretação, não etimologia): **virar** uma função e **rolá-la** sobre a outra.

con + *volvere*

“junto” “rolar”

⇓

convolutio

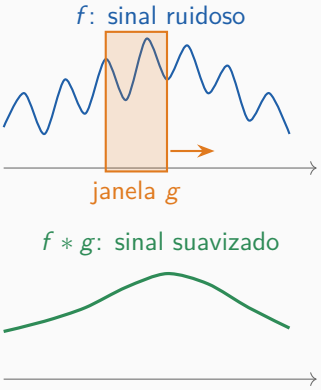
“enrolamento”

Lewis e Short (1879), *A Latin Dictionary*, verbetes *convolvo* e *volvo*; *Oxford English Dictionary*, verbete *convolution*.

Intuição: “enrolar junto” lembra o gesto da fórmula: virar g e fazê-la rolar ao longo de f, acumulando o produto. É uma leitura mnemônica, não uma definição. (etimologia: Lewis e Short, 1879)

1.2 O que é convolução?

- Uma operação que combina duas funções f e g numa terceira, $f * g$.
- Intuição: **soma ponderada deslizante**; o kernel g diz *como pesar* cada vizinhança de f.
- O kernel define o efeito: pesos ≥ 0 que somam $1 \Rightarrow$ média ponderada (suaviza); pesos que somam $0 \Rightarrow$ resposta nula onde o sinal é constante (realça bordas).
- Terminologia: f é a *entrada*, g o *kernel* e a saída, o *mapa de características* (Goodfellow et al., 2016, seç. 9.1).



Intuição: medições ruidosas são suavizadas por uma média ponderada, que pode pesar mais as recentes; no caso particular da média móvel de 7 dias da pandemia, os 7 pesos são iguais a $1/7$. (Goodfellow et al., 2016, seç. 9.1)

1.3 Convolução como média ponderada

- Um laser mede a posição de uma nave, com ruído: $x(t)$ é a posição medida no instante t.
- Para estimar a posição agora, faço uma média ponderada das medições, pesando mais as recentes.
- Uma medição feita no instante a tem idade $t - a$ e recebe peso $w(t - a)$.
- Fazendo isso em todo instante, saio com uma estimativa suavizada $s(t)$. Isso é uma convolução.

$$s(t) = \int x(a)\, w(t - a)\, da \qquad (9.1)$$

$$s(t) = (x * w)(t) \qquad (9.2)$$

Restrições do exemplo, não da definição

w deve ser uma densidade (senão não é média) e $w(a) = 0$ para $a < 0$ (não olha o futuro). Em geral, a convolução vale para quaisquer funções em que a integral exista.

Terminologia: $x =$ **entrada**, $w =$ **kernel**, $s =$ **mapa de características**.

Intuição: cada estimativa olha para trás e pondera as medições pela idade: o kernel diz quanto cada medição antiga ainda conta. (Goodfellow et al., 2016, seç. 9.1, pp. 327–328)

$$(f * g)(t) \;=\; \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

- Esta é a definição geral.
- O τ (variável de integração) percorre o eixo; o t é onde estou avaliando o resultado.
- O termo $g(t-\tau)$ é o g **espelhado**, por causa do menos τ, e **deslocado** até t.
- Multiplico pelo f e integro: sai um número para cada t. A integral existe (quase em toda parte), p.ex., se $f, g \in L^1(\mathbb{R})$.

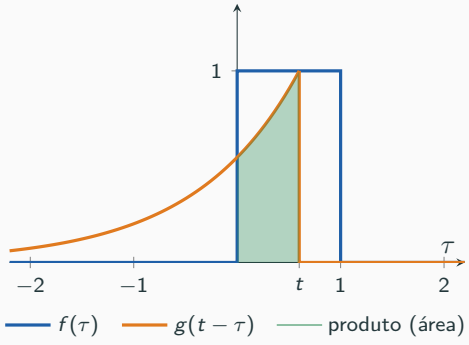
Regra mnemônica: **espelhar** → **deslocar** → **multiplicar** → **integrar**

Oppenheim e Willsky (1997), seç. 2.2. Em Goodfellow et al. (2016, eq. 9.1): $s(t) = \int x(a)\, w(t-a)\, da$.

Intuição: duas transparências: vire g ao contrário, deslize-a sobre f e, em cada posição t, a área sob o produto das curvas é o valor de $(f * g)(t)$. (Oppenheim e Willsky, 1997, seç. 2.2)

1.5 Convolução em tempo contínuo: um exemplo

Exemplo: $f(\tau) =$ pulso em $[0,1]$, $\quad g(\tau) = e^{-\tau}u(\tau)$, $u =$ degrau $\quad$ (avaliando em $t = 0{,}6$)



1. **Espelhar**: $g(\tau) \to g(-\tau)$.
2. **Deslocar**: $g(-\tau) \to g(t-\tau)$.
3. **Multiplicar**: $f(\tau)\, g(t-\tau)$.
4. **Integrar**: a área verde é $(f * g)(t)$.

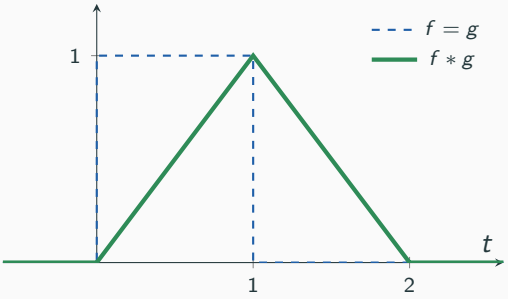
Para $0 \le t \le 1$:

$$(f * g)(t) = \int_0^t e^{-(t-\tau)}\, d\tau = 1 - e^{-t}$$

Em $t = 0{,}6$: $1 - e^{-0{,}6} \approx 0{,}45$.

Intuição: $g(t-\tau)$ pondera o passado: o que entrou há Δ unidades de tempo pesa $e^{-\Delta}$. A área verde é a entrada acumulada até t, descontada por essa memória. (Oppenheim e Willsky, 1997, seç. 2.2)

1.6 Exemplo clássico: pulso $*$ pulso $=$ triângulo



$$(f*f)(t) = \begin{cases} t, & 0 \le t \le 1 \\ 2-t, & 1 \le t \le 2 \\ 0, & \text{caso contrário} \end{cases}$$

- Deslizando um pulso sobre outro, a área de sobreposição cresce linearmente, chega ao máximo quando os dois coincidem, em $t = 1$, e cai. Resultado: um triângulo de 0 a 2.
- Suporte: $\operatorname{supp}(f*g) \subseteq \operatorname{supp} f + \operatorname{supp} g$; aqui, $[0,1] + [0,1] = [0,2]$.

- Um sensor real mede em intervalos regulares (p.ex., 1 vez por segundo): t passa a ser inteiro.

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)\, w(t-a) \qquad (9.3)$$

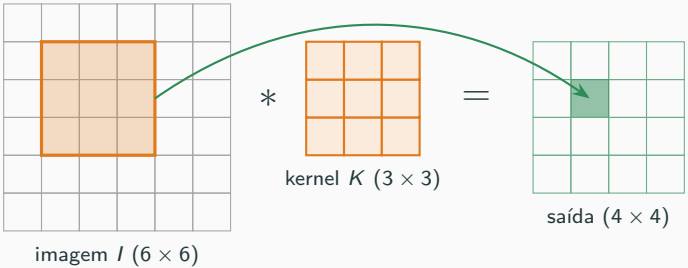
- Em aprendizado de máquina, a entrada é um tensor de dados e o kernel, um tensor de parâmetros aprendidos.
- Só guardamos um número finito de valores; fora deles, entrada e kernel valem zero. Por isso, a soma infinita vira, na prática, uma **soma finita**.

Intuição: a soma infinita é só notação: na prática, o kernel tem poucos pesos não nulos, e a conta percorre só eles. (Goodfellow et al., 2016, seç. 9.1, p. 328)

1.8 De 1D para 2D: o kernel desliza sobre a imagem

▶ 06_kernel_2d_curto

$$(I * K)(i,j) \;=\; \sum_{m}\sum_{n} I(i-m,\, j-n)\; K(m,n)$$



Cada pixel de saída = soma ponderada da vizinhança coberta pelo kernel; sem padding (*valid*), a saída tem $(6-3+1) \times (6-3+1) = 4 \times 4$ pixels. (Goodfellow et al., 2016, eqs. 9.4–9.5)

Intuição: como passar uma lupa pela imagem: em cada posição, a janela 3×3 é resumida por um único número, a soma ponderada pelos pesos do kernel. (Goodfellow et al., 2016, seç. 9.1)

1.9 Convolução em 2D: duas fórmulas equivalentes

Em imagens, a convolução corre em dois eixos. Por comutatividade, há duas formas:

$$S(i,j) = \underbrace{\sum_m \sum_n I(m,n)\, K(i-m, j-n)}_{\text{(9.4): } m,n \text{ indexam a imagem}} = \underbrace{\sum_m \sum_n I(i-m, j-n)\, K(m,n)}_{\text{(9.5): } m,n \text{ indexam o kernel}}$$

Na (9.4), com kernel 3×3 (índices $0, 1, 2$): só contribuem $m \in \{i-2,\ i-1,\ i\}$ e $n \in \{j-2,\ j-1,\ j\}$. O intervalo muda a cada pixel de saída (i,j), e ainda é cortado nas bordas da imagem.

Na (9.5), o intervalo é sempre o mesmo: $m, n \in \{0, 1, 2\}$, os índices do kernel. O laço fica fixo, para qualquer (i,j):

```
for m in 0..2:  for n in 0..2:
  S[i,j] += I[i-m,j-n]*K[m,n]
```

A igualdade sai da troca de variável $m' = i - m$, $n' = j - n$.

Intuição: na (9.5), o laço percorre o kernel, que é pequeno e fixo; na (9.4), percorreria a parte da imagem sob o kernel, que muda a cada posição. (Goodfellow et al., 2016, seç. 9.1, p. 328)

1.10 Espelhar ou não: a correlação cruzada

- Na (9.5), quando *m cresce*, o índice da imagem $i - m$ *decresce*: o kernel é percorrido espelhado.
- O espelhamento dá à convolução propriedades algébricas, como a comutatividade (e a associatividade): úteis em provas, irrelevantes na implementação.
- Por isso, as bibliotecas calculam a correlação cruzada, sem espelhar, e a chamam de "convolução".

Convolução:

$$S(i,j) = \sum_m \sum_n I(i-m, j-n)\, K(m,n)$$

Correlação cruzada (9.6):

$$S(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

A única diferença: $-$ vira $+$.

E isso importa numa rede?

Não: com pesos aprendidos, a rede simplesmente aprende o kernel espelhado. E, com ReLU e pooling, ela já não comuta.

Intuição: é a diferença entre ler o kernel da esquerda para a direita ou da direita para a esquerda; como a rede escolhe os pesos, ela se adapta a qualquer uma das leituras. (Goodfellow et al., 2016, seç. 9.1, p. 329)

1.11 Um cálculo à mão — e o detalhe do espelhamento

Vizinhança P

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel Sobel G_x

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Correlação (sem espelhar):

$$\sum P \odot K = 2 + 4 + 2 = 8$$

Convolução (kernel girado 180°):

$$\sum P \odot K^{\circlearrowleft} = -8$$

Atenção

PyTorch `Conv2d` e TensorFlow calculam a **correlação cruzada**, $S(i,j) = \sum_{m,n} I(i+m, j+n)\, K(m,n)$. Como os pesos são aprendidos, a rede aprende o kernel já espelhado: os modelos são equivalentes. Para kernels fixos (Sobel), a distinção importa. (Goodfellow et al., 2016, eq. 9.6; Zhang et al., 2023, seç. 7.2)

Intuição: girar o kernel 180° troca esquerda com direita e cima com baixo. Este Sobel é antissimétrico ($K^{\circlearrowleft} = -K$), por isso só o sinal muda; em geral, muda mais. (Goodfellow et al., 2016, seç. 9.1)

2. Motivação (cap. 9.2)

1. **Interações esparsas**: cada saída depende só de uma vizinhança $K \times K$.
2. **Compartilhamento de parâmetros**: o mesmo kernel é usado em todas as posições.
3. **Equivariância à translação**: deslocar a entrada desloca a saída.

$$\mathrm{conv}(T(x)) = T(\mathrm{conv}(x)), \quad T : \text{translação}$$

Não vale para rotação nem escala. Bordas e stride a tornam só aproximada (Azulay e Weiss, 2019).

Quanto se economiza

Imagem 28×28, saída do mesmo tamanho:

camada densa: $\approx$ **615 mil pesos**

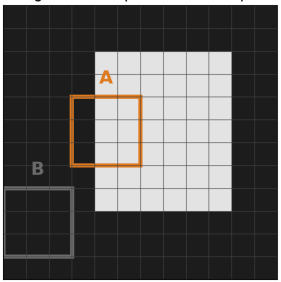
convolução 3×3: **9 pesos**

Densa: $784 \times 784 = 614\,656$; convolução: os mesmos 9 pesos em todas as posições.

Goodfellow et al. (2016), seç. 9.2; Zhang et al. (2023), seç. 7.1.

Intuição: uma borda se decide pela vizinhança (esparsidade); um detector útil num lugar é útil em todos (compartilhamento); se o objeto se desloca, a resposta o acompanha. (Goodfellow et al., 2016, seç. 9.2)

2.2 Interações esparsas: basta olhar a vizinhança



o detector só precisa de 9 pixels

A: sobre a borda → resposta 28

1	8	8
1	8	8
1	8	8

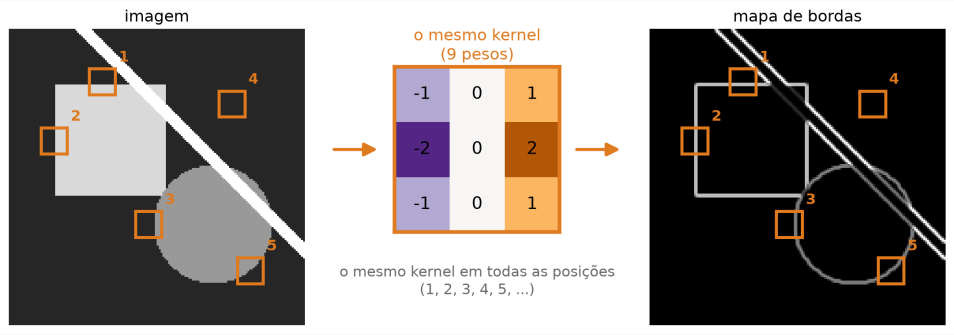
B: região lisa → resposta 0

1	1	1
1	1	1
1	1	1

- Cada saída depende só de uma vizinhança da entrada: com o Sobel G_x, a janela 3×3 sobre a borda (A) dá 28; a da região lisa (B), 0.
- Com m entradas e n saídas, a camada densa usa $m \times n$ pesos e $O(m \times n)$ operações; com $k \ll m$ ligações por saída, $k \times n$ e $O(k \times n)$.

Intuição: para ver se há um degrau à frente, basta olhar o chão perto dos pés, não a sala inteira: a informação sobre uma borda está na vizinhança dela. (Goodfellow et al., 2016, seç. 9.2, pp. 330–331)

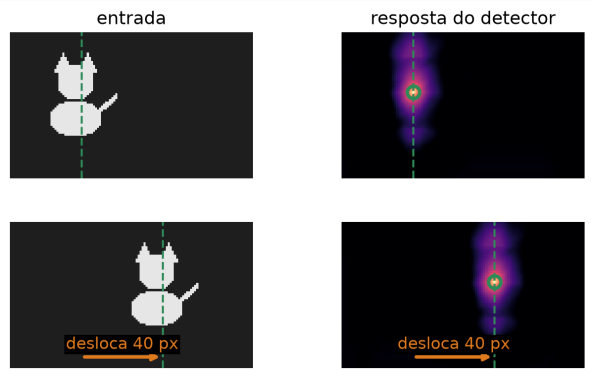
2.3 Compartilhamento de parâmetros: um detector, todas as posições



Um detector de bordas útil num canto da imagem é útil em qualquer canto: basta aprendê-lo uma vez e reaproveitá-lo em todas as posições. São 9 pesos para a imagem inteira, de qualquer tamanho.

Goodfellow et al. (2016), seç. 9.2.

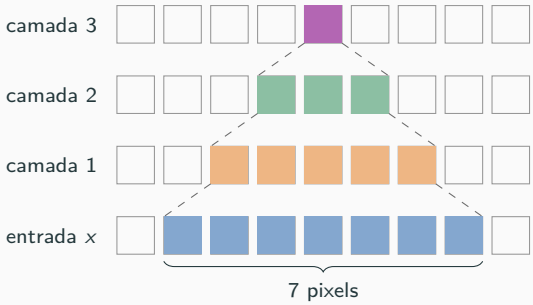
2.4 Equivariância à translação: o gato anda, a resposta anda junto



Se o gato anda 40 pixels para a direita, o pico do detector anda os mesmos 40 pixels: deslocar e convoluir dá o mesmo que convoluir e deslocar. Não vale para rotação nem escala.

Goodfellow et al. (2016), seç. 9.2. Detector: correlação com um modelo do gato.

2.5 Campo receptivo



- **Campo receptivo**: a região da entrada que influencia um neurônio.
- Cada camada (kernel 3) olha 3 vizinhos, que já olhavam outros 3: a região cresce 2 pixels por camada. Após L camadas: $R = 1 + L\,(K - 1) = 1 + 3 \cdot 2 = 7$.
- Camadas profundas "enxergam" regiões grandes com kernels pequenos.
- Redes treinadas: bordas $\rightarrow$ texturas $\rightarrow$ partes $\rightarrow$ objetos (Zeiler e Fergus, 2014).

Intuição: notícia passada de vizinho em vizinho: cada um só fala com quem está do lado, mas, após algumas rodadas, a informação já veio de longe. (Goodfellow et al., 2016, seç. 9.2, fig. 9.4)

2.6 Aplicação: por que a VGG usa só kernels 3×3

VGG (Simonyan e Zisserman, 2015): CNN do *Visual Geometry Group* de Oxford, entre as melhores do desafio ImageNet 2014. Sua ideia central: em vez de kernels grandes, empilhar várias convoluções 3×3.

Mesmo campo receptivo: 7×7

Pelo slide 2.5, 3 camadas 3×3 enxergam 7×7 pixels, o mesmo que 1 camada 7×7.

Quantos pesos? (C canais de entrada e de saída)

1 camada $K \times K$ tem $K^2 C^2$ pesos.
$3 \times (3 \times 3)$: $3 \cdot 9\, C^2 = 27\, C^2$
$1 \times (7 \times 7)$: $49\, C^2$

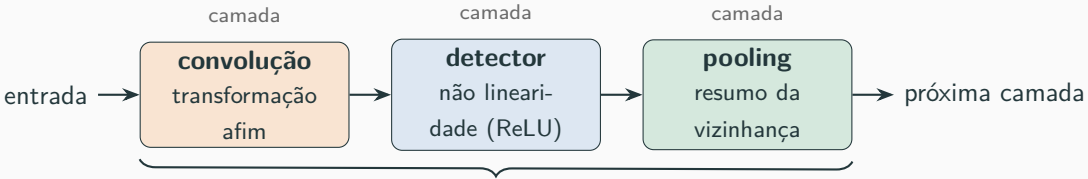
Resultado: **45% menos pesos** e **três não linearidades** (ReLU) em vez de uma, o que deixa a função mais expressiva.

Simonyan e Zisserman (2015), seç. 2.3; Zhang et al. (2023), seç. 8.2.

Intuição: três degraus pequenos levam à mesma altura de um degrau grande, com menos material, e cada degrau é uma chance de mudar de direção. (Simonyan e Zisserman, 2015)

3. Pooling (cap. 9.3)

terminologia "simples": cada estágio é uma camada (nem toda camada tem parâmetros)



terminologia "complexa": os três juntos formam *uma* camada convolucional

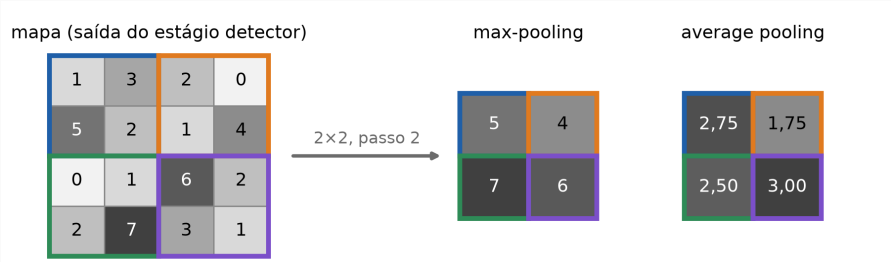
$$\text{detector}_o(i,j) = \sigma\big(b_o + \textstyle\sum_c \sum_{m,n} X_c(i+m, j+n)\, K_{o,c}(m,n)\big)$$

Sem σ, empilhar convoluções equivaleria a uma única convolução (associatividade).

Intuição: uma linha de montagem com três postos: medir cada vizinhança com o kernel, deixar passar só as respostas positivas e, por fim, resumir cada região. (Goodfellow et al., 2016, seç. 9.3, fig. 9.7)

3.2 Pooling: um resumo da vizinhança

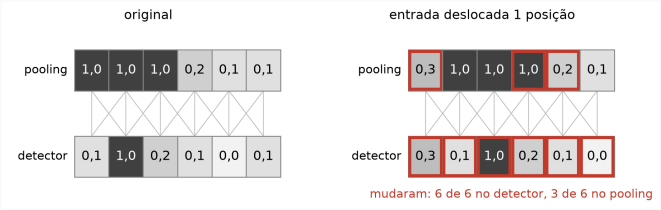
▶ 15_pooling_curto



A função de pooling substitui a saída da rede numa posição por uma **estatística-resumo** das saídas vizinhas: o **máximo** (*max pooling*), a **média** de uma vizinhança retangular, a **norma** L^2 da vizinhança ou uma média ponderada pela distância ao pixel central. Não tem parâmetros a aprender.

Intuição: a fase de grupos de um torneio: de cada grupo de 4, só o primeiro avança. Sabe-se quem foi forte em cada grupo, mas não em que posição jogou. (Goodfellow et al., 2016, seç. 9.3)

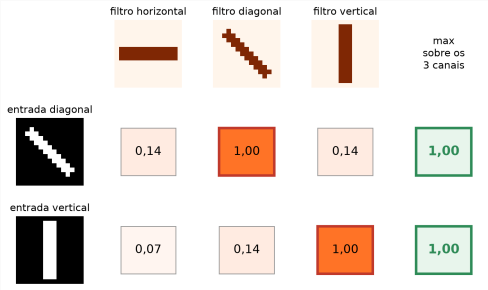
3.3 Invariância a pequenas translações



- **Invariância**: com a entrada deslocada de uma posição, todas as saídas do detector mudam, mas só *metade* das do pooling (a convolução é *equivariante*: a saída se desloca junto).
- Útil quando importa mais *se* a característica existe do que *onde*: num rosto, basta um olho à esquerda e outro à direita.
- Nem sempre: para achar um canto formado por duas bordas, a posição exata importa.
- O pooling impõe a invariância como certeza (um *prior* infinitamente forte; ver 4.1): se a tarefa depende da posição exata, pode causar *underfitting*.

Intuição: um alarme de incêndio por cômodo: dispara com fumaça em qualquer canto, e mover a fumaça um pouco não muda nada; mas não diz em que canto ela está. (Goodfellow et al., 2016, seç. 9.3–9.4)

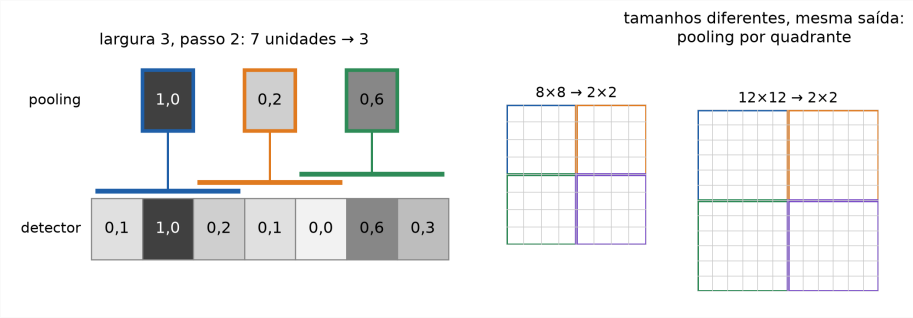
3.4 Pooling entre canais: invariância aprendida à rotação



- Até aqui, o pooling resumia posições vizinhas de *um* mapa. Ele também pode resumir *vários mapas*, um por filtro.
- Três filtros procuram o mesmo traço em orientações diferentes. Cada número é o cosseno entre entrada e filtro: aqui, de 0 (nada) a 1 (idêntica), pois não há valores negativos.
- Cada entrada ativa um filtro diferente, mas o máximo dos três é igual: a saída não muda quando o traço gira.
- Sozinha, a convolução não é invariante à rotação; com esse pooling, fica aproximadamente invariante às rotações cobertas pelos filtros (ideia das redes *maxout*).

Intuição: três vigias, cada um olhando numa direção: o alarme toca se qualquer um deles vir algo, não importa de que lado veio. (Goodfellow et al., 2016, seç. 9.3, fig. 9.9; Goodfellow et al., 2013)

3.5 Pooling com passo e entradas de tamanho variável

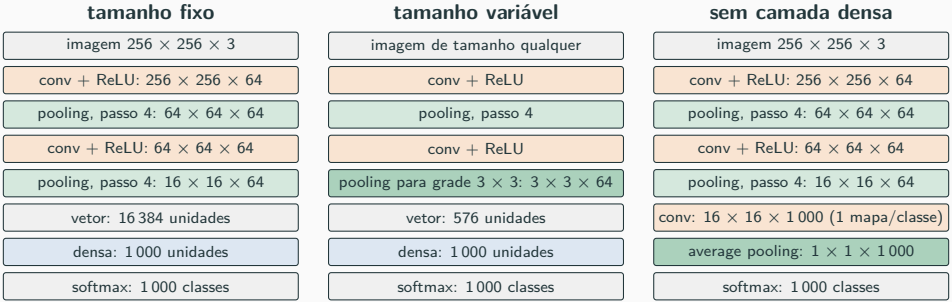


Reduzir a resolução: com regiões espaçadas de k pixels, a camada seguinte recebe cerca de k vezes menos entradas por dimensão (k^2 numa imagem): menos memória e menos computação.

Tamanho variável: ajustando o espaçamento das regiões, o classificador recebe sempre o mesmo número de resumos; p.ex., um por quadrante, qualquer que seja o tamanho da imagem.

Intuição: resumir um livro em um parágrafo por capítulo: livros de tamanhos diferentes viram resumos do mesmo formato. (Goodfellow et al., 2016, seç. 9.3, fig. 9.10)

3.6 Do pooling à arquitetura: três exemplos



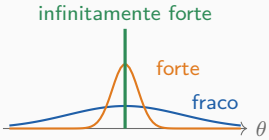
convolução pooling densa. No centro, o pooling garante o mesmo tamanho para qualquer entrada; à direita, o average pooling substitui a camada densa.

Intuição: convolução e pooling se alternam: cada par extrai padrões e encolhe a imagem; no fim, um classificador decide. Redes reais são maiores e mais ramificadas. (Goodfellow et al., 2016, seç. 9.3, fig. 9.11)

4. Convolução e pooling como um prior infinitamente forte (cap. 9.4)

4.1 O que é um *prior* infinitamente forte

Prior: distribuição de probabilidade sobre os parâmetros que codifica o que achamos razoável *antes* de ver os dados.



Infinitamente forte: probabilidade *zero* para alguns valores, que os dados não conseguem mudar.

- **Convolução** = camada densa com um prior infinitamente forte: pesos *zero* fora do campo receptivo e *iguais* entre posições. A função só pode aprender interações locais e equivariantes à translação.
- **Pooling** = prior infinitamente forte de *invariância* a pequenas translações.
- Ou seja: essas propriedades não são aprendidas; são impostas pela arquitetura.

Intuição: antes de ver qualquer foto, a CNN já "acredita" que o que importa é local e pode estar em qualquer lugar; nenhum dado consegue tirar essa crença dela. (Goodfellow et al., 2016, seç. 9.4)

4.2 Consequências do *prior* da convolução

1. Pode causar *underfitting*

Como todo prior, só ajuda se a suposição for razoável.

- Se a tarefa depende da **posição exata**, o pooling pode aumentar o erro de *treino*. Algumas redes (Szegedy et al., 2014) aplicam pooling só em parte dos canais: umas características invariantes, outras precisas.
- Se a tarefa depende de **pontos muito distantes** da entrada, o prior local da convolução pode ser inadequado.

2. Só compare CNNs com CNNs

Um modelo sem convolução aprenderia igual se os pixels fossem embaralhados (sempre do mesmo jeito): ele é *invariante a permutação* e precisa descobrir a topologia sozinho. A CNN já recebe pronta a noção de vizinhança. Por isso, há *benchmarks* separados para os dois tipos.

Intuição: um prior forte é um atalho: se a suposição vale, aprende-se com menos dados; se não vale, o atalho leva ao lugar errado. (Goodfellow et al., 2016, seç. 9.4)

5. Variantes da função de convolução básica (cap. 9.5)

5.1 Múltiplos canais de entrada e de saída

- Entrada com C_{in} canais $\Rightarrow$ cada filtro tem C_{in} fatias $K \times K$.
- Convolui canal a canal e soma, mais um viés:

$$Y_o = \sum_{c=1}^{C_{\text{in}}} X_c \star K_{o,c} + b_o$$

$\star$: correlação cruzada, como nas bibliotecas.

- C_{out} filtros $\Rightarrow$ C_{out} canais de saída.

Número de parâmetros

$$(K \cdot K \cdot C_{\text{in}} + 1) \cdot C_{\text{out}}$$

RGB, 3×3, 4 filtros: $(27 + 1) \cdot 4 = 112$.
Não depende do tamanho da imagem.

Goodfellow et al. (2016), seç. 9.5; Zhang et al. (2023), seç. 7.4.

Intuição: cada filtro é uma pergunta feita a todos os canais da vizinhança de uma vez ("há aqui uma transição de vermelho para verde?"); cada filtro gera um mapa. (Goodfellow et al., 2016, seç. 9.5)

- Não olha vizinhos: em cada pixel, mistura os canais.
- Equivale a uma camada densa aplicada pixel a pixel, com pesos compartilhados.
- Uso típico: reduzir ou aumentar o número de canais com pouco custo (NiN, GoogLeNet, ResNet).

Parâmetros

$$C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$$

$256 \to 64$: $256 \cdot 64 + 64 = 16\,448$.

Lin et al. (2014); Zhang et al. (2023), seç. 7.4.

Intuição: como misturar tintas: em cada pixel, os C_{in} valores são combinados linearmente, com proporções aprendidas e iguais em todos os pixels, sem olhar os vizinhos. (Lin et al., 2014)

5.3 Padding e stride

- **Padding** P: borda de zeros. Sem ela (*valid*), cada camada perde $K-1$ pixels por dimensão.
- **Stride** S: o kernel pula S posições; reduz a resolução.

Tamanho da saída

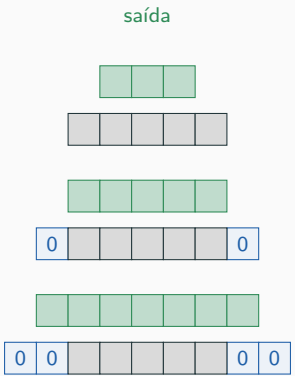
$$N_{\text{out}} = \left\lfloor \frac{N - K + 2P}{S} \right\rfloor + 1$$

N	K	P, S	N_{out}
5	3	0, 1	3
5	3	1, 1	5 (*same*)
5	3	1, 2	3
224	7	3, 2	112

Última linha: 1.ª camada da ResNet (He et al., 2016). Fórmula: Dumoulin e Visin (2016); Goodfellow et al. (2016), seç. 9.5; Zhang et al. (2023), seç. 7.3.

Intuição: padding é uma moldura de zeros para que o kernel também se centre nos pixels da borda; stride S avalia o kernel só a cada S posições: menos avaliações, saída menor. (Dumoulin e Visin, 2016)

5.4 Três tipos de *zero padding*: *valid*, *same* e *full*



valid: $N - K + 1 = 3$

same: $N = 5$

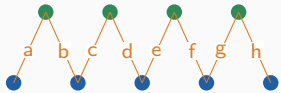
full: $N + K - 1 = 7$

- ***Valid*** (sem zeros): o kernel só é aplicado onde cabe inteiro. A cada camada, a saída perde $K - 1$ posições, o que limita a profundidade da rede.
- ***Same***: os zeros mantêm o tamanho, e a rede pode ter quantas camadas quiser. Porém, os pixels da borda influenciam menos saídas.
- ***Full***: cada pixel é visitado K vezes. Porém, as saídas da borda dependem de poucos pixels, e é difícil um mesmo kernel funcionar bem em todas as posições.
- Em geral, a quantidade de zeros que dá a melhor acurácia de teste fica entre o *valid* e o *same*.

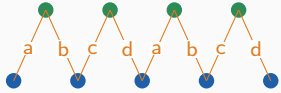
Intuição: sem moldura, a foto encolhe a cada carimbo; com uma moldura de zeros, ela mantém o tamanho. (Goodfellow et al., 2016, seç. 9.5, fig. 9.13)

5.5 Quanto compartilhar os pesos

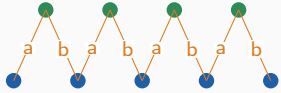
local: 8 pesos



tiled: 4 pesos



convolução: 2 pesos



4 saídas, cada uma ligada a 2 entradas; letras iguais = pesos iguais.

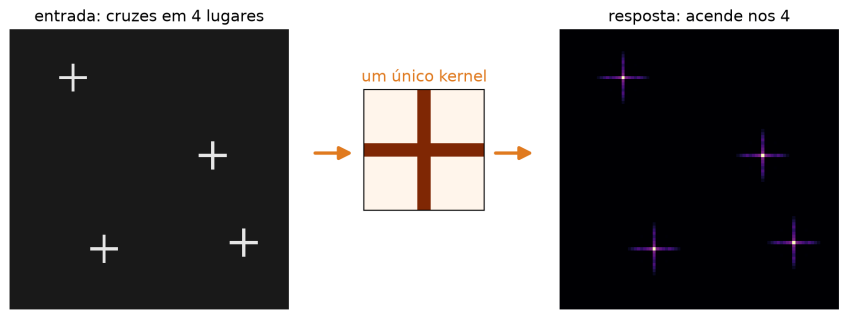
Nas três, cada saída olha só a sua vizinhança. O que muda é quantas posições compartilham os mesmos pesos:

- **localmente conectada**: nenhuma;
- ***tiled***: as posições se revezam entre t kernels;
- **convolução**: todas.

Quanto mais compartilhamento, menos pesos para aprender, e mais forte a suposição de que um padrão vale em qualquer lugar.

Intuição: convolução: o mesmo carimbo em toda a imagem; localmente conectada: um carimbo por lugar; *tiled*: um jogo de t carimbos que se alternam. (Goodfellow et al., 2016, seç. 9.5, fig. 9.16)

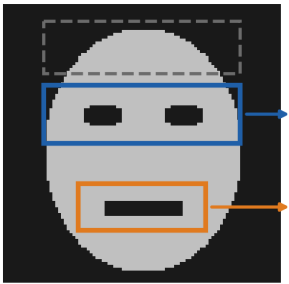
5.6 Convolução: um kernel para todas as posições



O mesmo kernel percorre a imagem inteira. Por isso, ele encontra o padrão (aqui, uma cruz) em qualquer posição, e o número de pesos não depende do tamanho da imagem. É a escolha certa quando o padrão pode aparecer em qualquer lugar.

Intuição: um único carimbo, procurado em toda a foto. (Goodfellow et al., 2016, seç. 9.2 e 9.5)

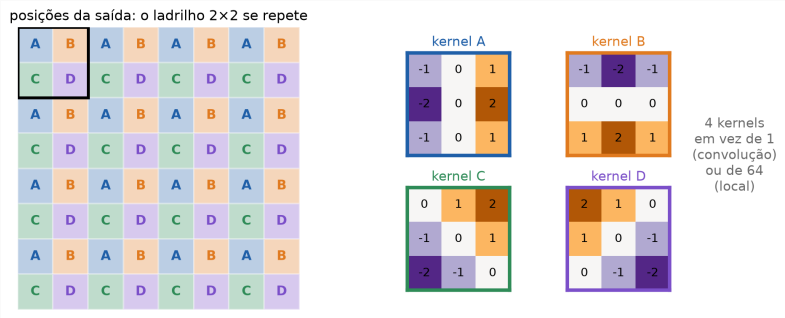
5.7 Localmente conectada: pesos próprios para cada posição



Também chamada de *unshared convolution*: cada saída olha só a sua vizinhança, mas cada posição tem pesos próprios. Faz sentido quando a imagem está alinhada e cada característica só aparece numa região. Num rosto centralizado, os olhos ficam sempre em cima e a boca, embaixo; não há por que procurar a boca na testa.

Intuição: um carimbo diferente para cada lugar da foto: cada região se especializa no que costuma aparecer nela. (Goodfellow et al., 2016, seç. 9.5, fig. 9.14)

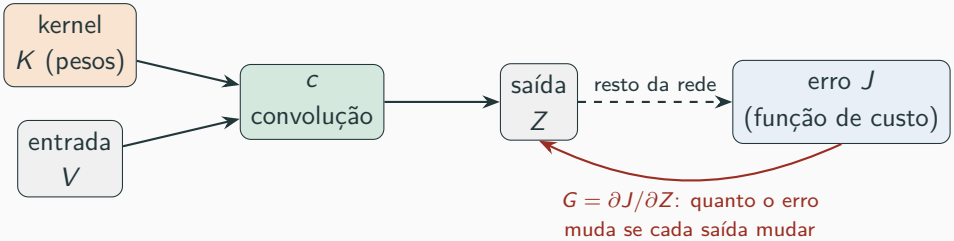
5.8 Convolução *tiled*: um jogo de kernels em rodízio



Aprende-se um conjunto de kernels, e cada posição usa um deles, seguindo um ladrilho que se repete. Com $t = 2$ em cada direção, são 4 kernels: posições vizinhas usam kernels diferentes, mas o padrão se repete a cada 2 casas. É um meio-termo entre a convolução e a camada localmente conectada.

Intuição: como o piso de ladrilhos: alguns desenhos diferentes, repetidos no mesmo padrão por todo o chão. (Goodfellow et al., 2016, seç. 9.5, fig. 9.16)

5.9 Retropropagação numa camada convolucional



Passagem direta: $Z = c(K, V, s)$, em que s é o passo (*stride*).

Passagem de volta (retropropagação): com G em mãos, a camada calcula:

- $\partial J/\partial K = g(G, V, s)$ (eq. 9.12): quanto o erro muda com cada peso; serve para atualizar o kernel.
- $\partial J/\partial V = h(K, G, s)$ (eq. 9.13): quanto o erro muda com cada entrada; é repassado à camada anterior.

Intuição: o erro chega de cima e se divide em duas partes: uma ajusta os pesos desta camada; a outra desce para a camada anterior, que faz o mesmo. (Goodfellow et al., 2016, seç. 9.5, eqs. 9.11–9.13)

5.10 Convolução transposta e viés

Convolução (direta): $z = W\,v$

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_5 \end{bmatrix}$$

Volta (operação h): $\dfrac{\partial J}{\partial v} = W^\top G$, de 3 valores para 5.

- As três operações, c, g e h, bastam para treinar qualquer rede convolucional *feedforward*.
- Na matriz do slide B.8, a operação h é a multiplicação pela transposta $W^\top$. Por isso, ela é chamada de *convolução transposta*.
- Como leva de menos para mais valores, ela também serve para aumentar a resolução, como na reconstrução de autoencoders.
- **Viés**: em geral, há um por canal de saída, o mesmo em todas as posições.

Intuição: a ida usa a matriz W; a volta, pelo mesmo caminho, usa a transposta $W^\top$. (Goodfellow et al., 2016, seç. 9.5)

5.11 Exemplo: aprendendo um kernel por gradiente

▶ 17_aprendendo_kernel_curto

- Dados X e o alvo $Y = X \star K_{\text{Sobel}}$, partimos de K aleatório.
- Minimizamos $L(K) = \frac{1}{M}\|X \star K - Y\|^2$ por descida de gradiente.
- L é quadrática e convexa em K; se a matriz das vizinhanças 3×3 de X tem posto 9, o mínimo é único: o Sobel (300 passos aqui).

O gradiente também é uma convolução

$$\nabla_K L = \frac{2}{M}\, X \star E, \qquad E = X \star K - Y$$

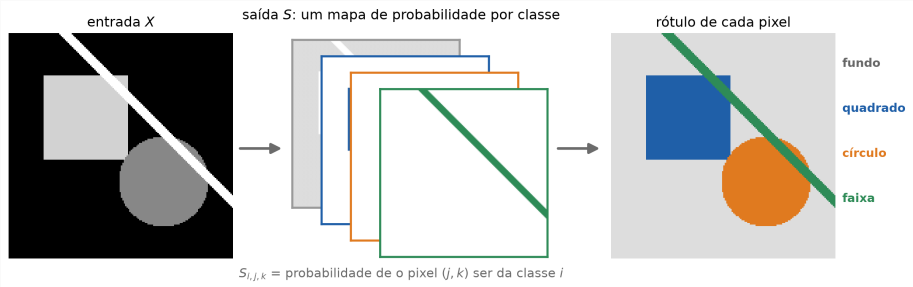
Correlação da entrada com o erro, como na passagem direta (Goodfellow et al., 2016, seç. 9.5).

Adaptado de Zhang et al. (2023), seç. 7.2.4, que aprende o kernel $[1, -1]$.

Intuição: como afinar um violão: mede-se o erro (alto ou baixo?) e gira-se a tarraxa um pouco no sentido que o reduz; o gradiente dá esse sentido aos 9 pesos de uma vez. (Goodfellow et al., 2016, seç. 4.3)

6. Saídas estruturadas (cap. 9.6)

6.1 Saídas estruturadas: um rótulo para cada pixel

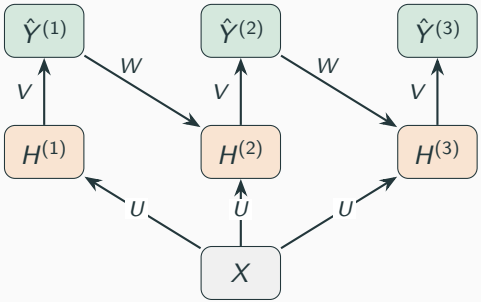


Uma CNN não precisa devolver só um rótulo para a imagem inteira: ela pode devolver um **objeto estruturado**, como um tensor S, em que $S_{i,j,k}$ é a probabilidade de o pixel (j, k) pertencer à classe i. Isso permite rotular cada pixel e desenhar o contorno de cada objeto (*segmentação*).

Problema: o pooling com passo grande deixa a saída menor que a entrada. **Soluções**: evitar o pooling, produzir uma grade de rótulos de resolução menor ou usar pooling com passo 1.

Intuição: em vez de dizer "nesta foto há um círculo", a rede pinta cada pixel com a cor da sua classe, como num livro de colorir. (Goodfellow et al., 2016, seç. 9.6)

6.2 Refinar o palpite: rede convolucional recorrente



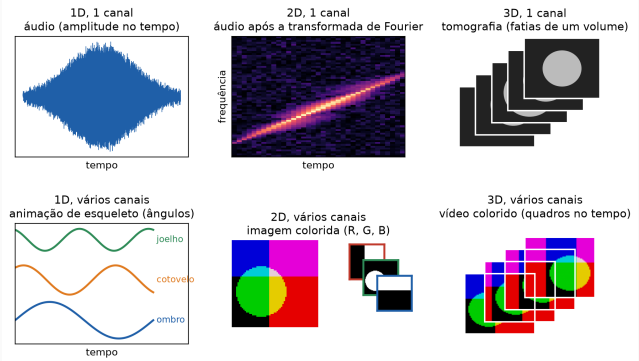
os mesmos kernels U, V e W em todos os passos

- A rede faz um primeiro palpite $\hat{Y}^{(1)}$ para o rótulo de cada pixel.
- Depois, refina esse palpite várias vezes, olhando a entrada X e o palpite anterior.
- Como os pesos são os mesmos em todos os passos, é uma rede recorrente feita de convoluções (Pinheiro e Collobert, 2014).
- Em seguida, um pós-processamento agrupa pixels vizinhos com o mesmo rótulo em regiões contíguas, p.ex., com modelos gráficos.

Intuição: como um esboço que se passa a limpo várias vezes: a cada passada, o desenho fica mais fiel à imagem. (Goodfellow et al., 2016, seç. 9.6, fig. 9.17)

7. Tipos de dados (cap. 9.7)

7.1 Tipos de dados: dimensões e canais



Os dados têm **dimensões** (tempo, espaço), ao longo das quais o kernel desliza, e **canais**: as grandezas medidas em cada ponto, que a convolução combina (seção 5.1). No espectrograma, convoluir também na frequência torna o modelo equivariante a deslocamentos de frequência.

Intuição: dimensões são as "direções" em que o kernel desliza; canais são quantos números há em cada ponto. (Goodfellow et al., 2016, seç. 9.7, tab. 9.1)

7.2 Entradas de tamanho variável



o mesmo kernel de 3 pesos (laranja), aplicado em entradas de tamanhos diferentes

O problema

Uma camada densa tem um peso para cada valor da entrada. Por isso, ela só aceita entradas de um tamanho fixo.

A solução da convolução

O kernel é aplicado em cada posição. Numa entrada maior, ele é aplicado mais vezes, e a saída cresce; os pesos continuam os mesmos.

Intuição: o mesmo carimbo serve para folhas de qualquer largura: numa folha maior, basta carimbar mais vezes. (Goodfellow et al., 2016, seç. 9.7)

7.3 Entradas de tamanho variável: quando a convolução se aplica

Aplica-se

observações do mesmo tipo



gravação de 2 s



gravação de 4 s

A entrada varia em tamanho porque contém *mais observações do mesmo tipo* ao longo de um eixo (tempo ou espaço). Toda posição tem a mesma semântica (amplitude num instante), o que justifica o *compartilhamento de parâmetros*: o mesmo kernel é aplicado em todas as posições.

Não se aplica

tipos distintos de observação

candidato A	
média na graduação	8,5
candidato B	
média na graduação	7,9
exame padronizado	9,0

A entrada de B é maior porque inclui uma observação *opcional* de outro tipo (nem todo candidato fez o exame). Cada posição corresponde a um atributo distinto, logo não faz sentido convoluir os mesmos pesos sobre a média e sobre o exame.

Intuição: a convolução pressupõe que a mesma característica pode ocorrer em qualquer posição; numa ficha de inscrição, cada posição é uma pergunta diferente. (Goodfellow et al., 2016, seç. 9.7)

8. Algoritmos eficientes de convolução (cap. 9.8)

8.1 Por que buscar algoritmos eficientes

Aplicações modernas usam redes com **mais de um milhão de unidades** (valores dos mapas de características), e calcular cada unidade exige uma soma ponderada (uma aplicação do kernel): o custo cresce com o número de *unidades*, e não só com o de parâmetros.

Ordem de grandeza. Uma única camada 3×3 da VGG, com 64 canais de entrada e 64 de saída, sobre um mapa de 224×224:

$$\underbrace{224 \cdot 224}_{\text{posições}} \cdot \underbrace{64}_{\text{canais de saída}} \cdot \underbrace{3 \cdot 3 \cdot 64}_{\text{pesos por saída}} \approx 1{,}85 \times 10^9 \text{ multiplicações por imagem.}$$

Essa camada tem $224 \cdot 224 \cdot 64 \approx 3{,}2$ milhões de unidades, mas só $3 \cdot 3 \cdot 64 \cdot 64 + 64 = 36\,928$ parâmetros, graças ao compartilhamento de parâmetros (2.3).

Caminho 1: hardware

Implementações que exploram computação paralela (GPUs). É essencial, mas não muda o número de operações.

Caminho 2: algoritmo

Reduzir o número de operações. Comparação em 8.2–8.3; cada método é detalhado nos slides de backup (B.1–B.5).

Goodfellow et al. (2016), seç. 9.8 e 12.1.2; Simonyan e Zisserman (2015).

8.2 Comparação de custo entre algoritmos

O exemplo é a camada do slide 8.1 (kernel 3×3, 64 canais), com $\approx 3{,}2$ milhões de valores de saída. No cálculo direto, cada valor é uma soma de $3 \cdot 3 \cdot 64 = 576$ produtos. (Contagens calculadas; Lavin e Gray, 2016; Howard et al., 2017.)

método	como economiza	mult./saída	ganho	resultado
direto	não economiza (referência)	576	$1\times$	exato
FFT (B.1–B.2)	convolução vira produto na frequência	≈ 210	$\approx 2{,}7\times$	exato
kernel separável (B.3)	3×3 vira 3×1 seguido de 1×3	384	$1{,}5\times$	exato*
Winograd $F(2\times2, 3\times3)$ (B.4)	troca produtos por somas	256	$2{,}25\times$	exato
Winograd $F(4\times4, 3\times3)$ (B.4)	idem, com blocos maiores	144	$4\times$	exato
separável em profundidade (B.5)	cada canal sozinho, depois 1×1	73	$7{,}9\times$	outra camada

- **ganho** $= 576 \div$ mult./saída. Na camada inteira ($\times$ 3,2 milhões): de $1{,}85 \times 10^9$ (direto) a $0{,}23 \times 10^9$ (em profundidade).
- *Se o kernel for separável (posto 1), como o Sobel. FFT e Winograd: exatos a menos de arredondamento; Winograd sem as transformadas.
- FFT: estimativa com preenchimento até 256×256. Com 64 canais, cada FFT de entrada serve a 64 saídas: o custo das transformadas se dilui (num só sinal 1D, B.2, a FFT perde).

8.3 Como ler a tabela: o que cada número esconde

Exatos

Direto, FFT, Winograd. Calculam *a mesma função*, só com outra ordem de contas. Trocar um pelo outro não muda o que a rede aprende; muda só a velocidade.

Exato só às vezes

Kernel separável. Vale apenas quando o kernel é um produto externo de vetores (Sobel, Gauss). Um kernel aprendido qualquer, em geral, não é.

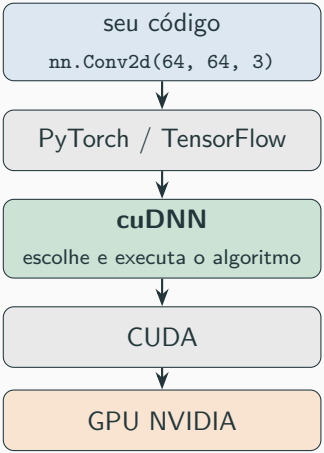
Muda o modelo

Separável em profundidade. É outra camada, com menos parâmetros, e a rede é treinada assim desde o início (MobileNet, Xception, ConvNeXt).

Custos que a contagem não mostra

- **FFT**: usa números complexos e guarda cada kernel transformado no tamanho do mapa ($64 \cdot 64$ kernels: $\approx 1{,}1\,\text{GB}$). Vence com kernels *grandes* (Mathieu et al., 2014), raramente com 3×3.
- **Winograd**: blocos maiores economizam mais, mas acumulam erro de arredondamento. Costuma ser o mais rápido para 3×3.
- **Em geral**: menos multiplicações não garante menos tempo; memória e paralelismo também pesam. A cuDNN (8.4) testa os algoritmos e escolhe o mais rápido para cada camada.

8.4 O que é a cuDNN (além do livro)



O que é. Biblioteca da NVIDIA (*CUDA Deep Neural Network library*) com rotinas otimizadas para GPU: convolução, pooling, normalização, ativações. Em GPUs NVIDIA, os frameworks não fazem a conta; chamam a cuDNN.

Por que importa aqui. A *mesma* convolução tem várias implementações: multiplicação de matrizes (*im2col* + GEMM, outra forma de escrever a convolução como produto de matrizes, diferente da do slide B.8), FFT, Winograd. A melhor depende do kernel, dos canais, do lote e da GPU.

Por isso a cuDNN pode **medir** cada opção na primeira execução e guardar a mais rápida para cada camada:

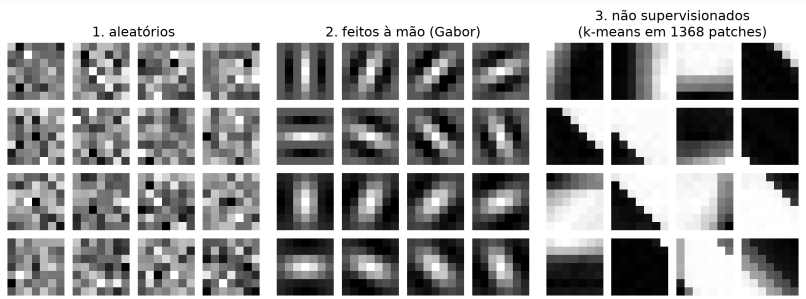
```
torch.backends.cudnn.benchmark = True
```

Intuição: como um GPS que testa as rotas possíveis e usa a mais rápida em cada trecho; o destino (o resultado) é o mesmo. (Chetlur et al., 2014)

9. Características aleatórias ou não supervisionadas (cap. 9.9)

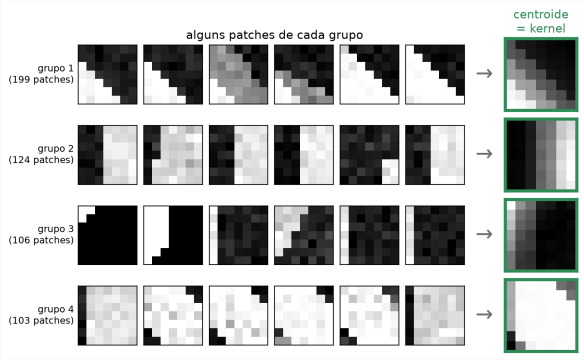
9.1 Kernels sem treino supervisionado: três estratégias

O mais caro no treino de uma CNN é **aprender as características** (os kernels); a camada de saída é barata, pois após o pooling recebe poucas características. Para reduzir o custo, os kernels podem ser obtidos *sem* treino supervisionado:



1. sorteados; **2.** projetados à mão, como detectores de borda numa orientação e escala (aqui, Gabor); **3.** aprendidos com critério *não supervisionado*: Coates et al. (2011) aplicam k-means a pequenos recortes (*patches*) e usam cada centroide como kernel (9.2). Figura calculada: 16 kernels 7×7 por estratégia; k-means sobre as imagens sintéticas dos slides anteriores.

9.2 k-means em patches: o que é um centroide



Calculado: 1368 patches, $k = 16$; 4 grupos mostrados.

Cada *patch* 7×7 vira um vetor de 49 números. O k-means divide os patches em k grupos, repetindo até estabilizar:

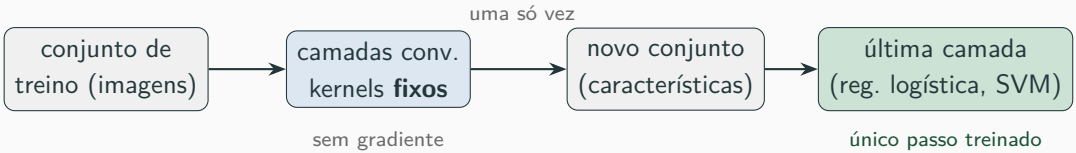
1. cada patch vai para o grupo do centroide mais próximo;
2. cada centroide vira a **média** do seu grupo: $\mu_j = \frac{1}{|G_j|} \sum_{p \in G_j} p$.

O **centroide** é o patch médio do grupo, a sua forma típica (uma borda vertical, uma diagonal...).

Como também é 7×7, serve **diretamente como kernel**: responde forte onde a imagem se parece com o grupo. Nenhum rótulo é usado.

Intuição: juntando centenas de fotos de rostos e calculando o "rosto médio", esse rosto médio é o centroide do grupo. (Coates et al., 2011; Bishop, 2006, seç. 9.1)

9.3 Características fixas: extrair uma vez, treinar só a última camada



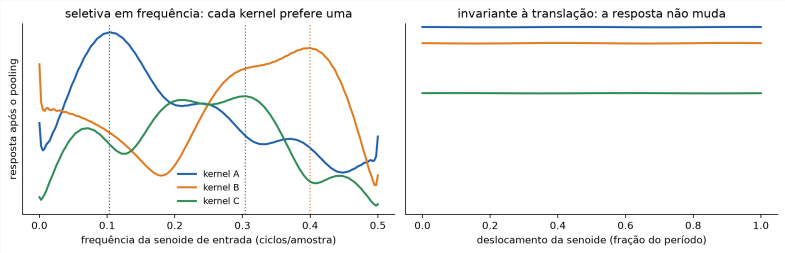
Por que é barato. Se as características não dependem do classificador, elas são determinadas *separadamente*. Basta passar o conjunto de treino pelas camadas convolucionais uma única vez e guardar as saídas.

Por que é mais fácil. Com as características fixas, treinar a última camada costuma ser um problema de otimização **convexo**, quando ela é uma regressão logística ou uma SVM: qualquer mínimo local é global, sem as dificuldades do treino de uma rede profunda.

Intuição: congela-se o "olho" da rede e treina-se só quem decide a partir do que ele vê. (Goodfellow et al., 2016, seç. 9.9)

9.4 Por que pesos aleatórios funcionam surpreendentemente bem

Filtros aleatórios muitas vezes funcionam surpreendentemente bem em CNNs (Jarrett et al., 2009; Pinto et al., 2011). Saxe et al. (2011) mostraram por quê: com pesos aleatórios, **convolução seguida de pooling** já é **seletiva em frequência** e **invariante à translação**.



Calculado: 3 kernels 1D aleatórios (9 pesos), entrada senoidal, pooling quadrático sobre toda a saída.

Uso prático: comparar *arquiteturas* treinando só a última camada de cada uma; treinar por completo só a melhor.

Intuição: boa parte do desempenho vem da arquitetura (convolução + pooling), não só dos valores dos pesos. (Saxe et al., 2011)

9.5 Pré-treinamento camada a camada e o que se faz hoje

Pré-treinamento guloso. Aprender as características sem propagar pela rede inteira a cada passo:

1. treinar a 1.ª camada isoladamente;
2. extrair suas features uma só vez;
3. treinar a 2.ª sobre elas, e assim por diante.

Exemplo canônico: a *convolutional deep belief network* (Lee et al., 2009).

Um passo além. Treina-se um modelo de um *patch* pequeno (como o k-means) e usam-se seus parâmetros como kernels: a rede é treinada sem usar convolução, e o custo alto fica só na inferência.

Popular de 2007 a 2013 (aprox.)

Conjuntos rotulados pequenos e poder computacional limitado.

Hoje

A maioria das CNNs é treinada de forma **puramente supervisionada**, com retropropagação pela rede inteira a cada iteração.

Questão em aberto

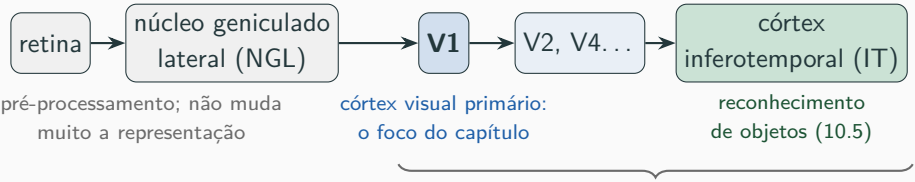
O ganho vem de *regularização*, ou só de permitir arquiteturas *maiores*, por ser mais barato? Ainda é difícil separar.

Goodfellow et al. (2016), seç. 9.9 e cap. 15; Coates et al. (2011); Lee et al. (2009).

10. A base neurocientífica das redes convolucionais (cap. 9.10)

10.1 Hubel e Wiesel: o ponto de partida

As redes convolucionais talvez sejam o maior caso de sucesso da IA inspirada na biologia. Parte central dessa inspiração vem de **Hubel e Wiesel** (1959, 1962, 1968; Nobel de 1981): registrando neurônios individuais do córtex visual de gatos, viram que muitos respondiam fortemente a padrões de luz muito específicos, como **barras com uma orientação precisa**, e quase nada a outros padrões.



Intuição: a CNN não copia o cérebro; copia alguns princípios de organização da V1, medidos em laboratório. (Goodfellow et al., 2016, seç. 9.10)

10.2 Três propriedades da V1 que a camada convolucional imita

propriedade da V1	o que significa	na CNN
1. mapa espacial	A V1 tem estrutura 2D que espelha a imagem na retina: luz na metade inferior da retina afeta só a metade correspondente da V1.	características organizadas em **mapas 2D**
2. células simples	A atividade é aproximadamente uma **função linear** da imagem num campo receptivo pequeno e localizado.	**unidades detectoras**: convolução (e não linearidade)
3. células complexas	Respondem a características parecidas com as das simples, mas são **invariantes a pequenos deslocamentos** dela (e a certas mudanças de iluminação).	**pooling**; e pooling entre canais (maxout, slide 3.4)

Os mesmos princípios parecem se repetir nas áreas seguintes: detecção e pooling, várias vezes.

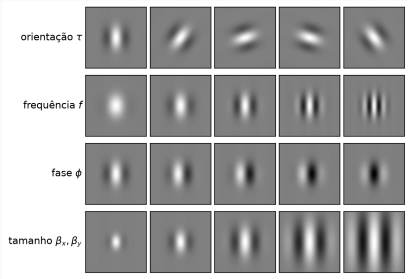
Intuição: simples ≈ "há uma borda vertical *exatamente aqui*"; complexa ≈ "há uma borda vertical *por perto*". (Goodfellow et al., 2016, seç. 9.10; Hubel e Wiesel, 1962)

10.3 Células simples: funções de Gabor

Como se mede. *Correlação reversa*: mostra-se ruído branco, registra-se a resposta do neurônio e ajusta-se um modelo linear; os pesos obtidos são o "kernel" do neurônio. Na maioria das células simples da V1, eles são bem descritos por uma **função de Gabor**:

$$w(x, y) = \alpha\, e^{-\beta_x x'^2 - \beta_y y'^2} \cos\big(f x' + \phi\big)$$

em que x', y' são as coordenadas giradas pelo ângulo τ em torno do centro (x_0, y_0).



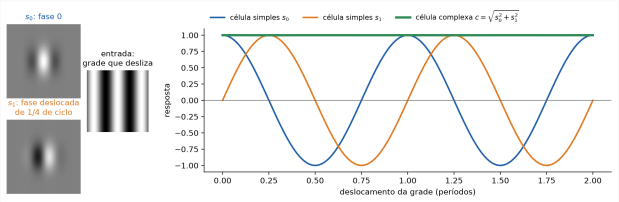
α: intensidade
f: frequência das listras
τ: orientação
β_x, β_y: tamanho (inverso)
ϕ: fase (onde caem as listras)
x_0, y_0: posição do campo

Intuição: gaussiana × cosseno: uma "janela" que limita *onde* a célula olha, e listras que dizem *o quê* ela procura. (Goodfellow et al., 2016, seç. 9.10, eq. 9.15–9.17)

10.4 Células complexas: o modelo de energia

Uma célula complexa pode ser modelada combinando duas células simples s_0 e s_1 com os mesmos parâmetros, exceto a fase, deslocada de 1/4 de ciclo (um *par em quadratura*):

$$c(I) = \sqrt{s_0(I)^2 + s_1(I)^2}$$



Calculado: com a grade, c fica constante (0,07%) e não muda com a imagem **negada** (só entram quadrados); em imagens quaisquer, ela é aproximada (pequenos deslocamentos na direção τ).

Intuição: é uma forma de pooling quadrático, parente da do slide 9.4, mas sobre dois filtros na mesma posição; na grade, $s_0^2 + s_1^2 = \cos^2 + \sin^2$ não depende do deslocamento. (Adelson e Bergen, 1985; Goodfellow et al., 2016, seç. 9.10)

10.5 Além da V1: “células avó” e o córtex inferotemporal

Células avó. Indo mais fundo no cérebro, encontram-se neurônios que respondem a um conceito específico e são invariantes a muitas transformações da entrada. O exemplo famoso é o **“neurônio Halle Berry”**, no lobo temporal medial de humanos: disparava para fotos da atriz, desenhos dela e até o nome escrito, e não para outras pessoas (Quiroga et al., 2005).

Eles são mais gerais que as CNNs do livro, que não generalizariam automaticamente para o nome escrito. Trabalhos posteriores acharam unidades parecidas em modelos multimodais, como o CLIP (Goh et al., 2021).

Córtex inferotemporal (IT). É o análogo mais próximo da *última camada de características* de uma CNN.

Para uma visão rápida do objeto, nos **primeiros $\sim$100 ms**, antes que o *feedback* de outras áreas pese:

- as CNNs **predizem as taxas de disparo** de neurônios do IT;
- e reconhecem objetos com desempenho semelhante ao de humanos com tempo limitado.

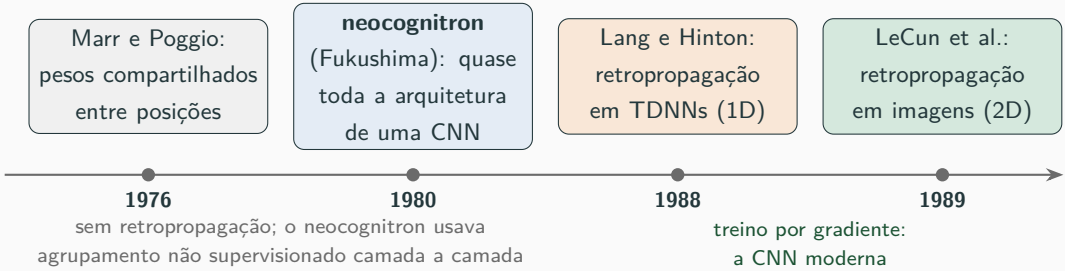
Intuição: a semelhança é maior na primeira olhada; depois, o cérebro faz muito mais do que uma passagem para a frente. (DiCarlo, 2013; Yamins et al., 2014)

10.6 Onde a analogia falha

- **Resolução.** O olho tem alta resolução só na **fóvea** (um polegar a um braço de distância) e compõe a cena com **movimentos sacádicos** e atenção. A CNN recebe a imagem inteira em alta resolução.
- **Integração.** A visão é integrada a outros sentidos, como a audição, e a fatores como humor e pensamento.
- **Tarefa.** O cérebro entende **cenas inteiras**: relações entre objetos e geometria 3D, não só rótulos.
- **Feedback.** Até a V1 é fortemente influenciada por áreas superiores; em redes, o feedback ainda não trouxe ganho convincente (até 2016, segundo o livro).
- **Neurônios.** Provavelmente usam outras ativações e outros poolings; um neurônio da V1 pode exigir **vários filtros quadráticos** (Rust et al., 2005), e não um só filtro linear.

Intuição: o córtex inferotemporal (IT, slide 10.5) e a CNN carregam informação parecida, mas os cálculos intermediários podem ser bem diferentes. (Goodfellow et al., 2016, seç. 9.10)

10.7 O que a neurociência não disse: como treinar

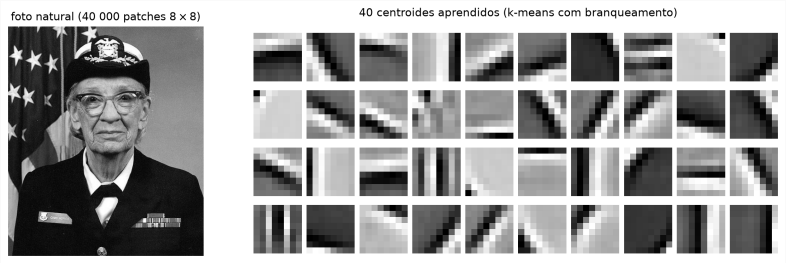


A neurociência forneceu a **arquitetura** (mapas, detecção local, pooling), mas pouco disse sobre **como treinar**. O ingrediente que faltava, o aprendizado por gradiente com retropropagação, veio de outra linha de pesquisa.

Intuição: a biologia deu a planta da casa; o método para construí-la veio da matemática. (Goodfellow et al., 2016, seç. 9.10)

10.8 Por que tantos algoritmos aprendem “Gabor”

Em **imagens naturais**, muitos algoritmos aprendem sozinhos características parecidas com Gabor, como as da V1: codificação esparsa (Olshausen e Field, 1996), a 1.ª camada de redes profundas e também o k-means, desde que com branqueamento e em imagens naturais.

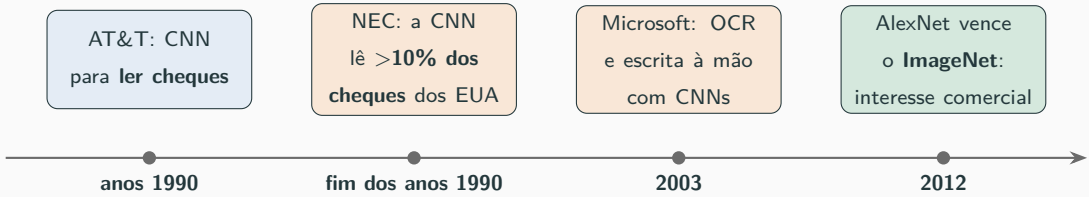


Calculado: k-means com branqueamento (Coates e Ng, 2012) numa foto de domínio público (Grace Hopper, Marinha dos EUA): surgem bordas **localizadas e orientadas**. Por isso não dá para dizer qual algoritmo é “o” modelo do cérebro; mas um que *não* aprendesse bordas seria mau candidato.

Intuição: bordas orientadas fazem parte da estatística das imagens naturais; quem aprende bem com elas as encontra. (Hyvärinen et al., 2009; Goodfellow et al., 2016, seç. 9.10)

11. Redes convolucionais e a história do aprendizado profundo (cap. 9.11)

11.1 As CNNs abriram o caminho



- As CNNs são um exemplo bem-sucedido de ideias tiradas do estudo do cérebro (seção 10).
- Estiveram entre os **primeiros modelos profundos a funcionar bem**, muito antes de redes profundas arbitrárias serem consideradas viáveis, e entre as primeiras treinadas com sucesso por retropropagação.
- Também foram das primeiras a resolver **aplicações comerciais** importantes, e já venciam competições anos antes de 2012.

Goodfellow et al. (2016), seç. 9.11; LeCun et al. (1989, 1998); Simard et al. (2003); Krizhevsky et al. (2012).

11.2 Por que funcionaram antes, e o que vem depois

Por que as CNNs deram certo quando as redes densas com retropropagação eram tidas como um fracasso? Não está claro. Hipóteses do livro:

- **Eficiência**: com menos operações e parâmetros (seções 2 e 8), dava para rodar mais experimentos e ajustar implementação e hiperparâmetros.
- **Tamanho**: redes maiores parecem mais fáceis de treinar. Com hardware moderno, redes densas grandes têm desempenho razoável em muitas tarefas, mesmo com os dados e as ativações da época em que se achava que elas não funcionavam.
- **Barreira psicológica**: talvez os praticantes não esperassem que redes neurais funcionassem e não se esforçassem a sério.

O papel delas

As CNNs "carregaram a tocha" do aprendizado profundo e abriram caminho para a aceitação das redes neurais em geral.

O que vem depois

Elas especializam as redes para dados com **topologia de grade** (sobretudo imagens 2D) e permitem escalá-las. Para **sequências 1D**, o livro passa a outra especialização: as **redes recorrentes** (cap. 10).

Intuição: as CNNs estiveram entre as primeiras a provar que uma rede profunda bem estruturada podia funcionar na prática. (Goodfellow et al., 2016, seç. 9.11)

Referências i

- I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 9.
- C. T. Lewis, C. Short. *A Latin Dictionary*. Oxford: Clarendon Press, 1879.
- *Oxford English Dictionary*. Oxford University Press. Verbete “convolution”.
- A. Zhang, Z. C. Lipton, M. Li, A. J. Smola. *Dive into Deep Learning* (D2L). Cambridge University Press, 2023. Caps. 7 e 8. `https://d2l.ai`
- A. V. Oppenheim, A. S. Willsky, S. H. Nawab. *Signals and Systems*. 2.ª ed. Prentice Hall, 1997.
- A. V. Oppenheim, R. W. Schafer. *Discrete-Time Signal Processing*. 3.ª ed. Pearson, 2010.
- R. C. Gonzalez, R. E. Woods. *Digital Image Processing*. 4.ª ed. Pearson, 2018.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*. 3.ª ed. MIT Press, 2009.

Referências ii

- V. Dumoulin, F. Visin. A guide to convolution arithmetic for deep learning. arXiv:1603.07285, 2016.
- M. Mathieu, M. Henaff, Y. LeCun. Fast training of convolutional networks through FFTs. ICLR, 2014.
- A. Coates, A. Ng, H. Lee. An analysis of single-layer networks in unsupervised feature learning. AISTATS, 2011.
- A. Coates, A. Y. Ng. Learning feature representations with k-means. In: *Neural Networks: Tricks of the Trade*, 2.ª ed., Springer, 2012.
- D. H. Hubel, T. N. Wiesel. Receptive fields of single neurones in the cat's striate cortex. *J. Physiol.*, 148:574–591, 1959.
- D. H. Hubel, T. N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *J. Physiol.*, 160:106–154, 1962.

Referências iii

- D. H. Hubel, T. N. Wiesel. Receptive fields and functional architecture of monkey striate cortex. *J. Physiol.*, 195:215–243, 1968.
- E. H. Adelson, J. R. Bergen. Spatiotemporal energy models for the perception of motion. *JOSA A*, 2(2):284–299, 1985.
- G. Goh et al. Multimodal neurons in artificial neural networks. *Distill*, 2021.
- R. Q. Quiroga, L. Reddy, G. Kreiman, C. Koch, I. Fried. Invariant visual representation by single neurons in the human brain. *Nature*, 435:1102–1107, 2005.
- J. J. DiCarlo. Mechanisms underlying visual object recognition: humans vs. neurons vs. machines. Tutorial, NIPS, 2013.
- D. L. K. Yamins et al. Performance-optimized hierarchical models predict neural responses in higher visual cortex. *PNAS*, 111(23):8619–8624, 2014.

Referências iv

- N. C. Rust, O. Schwartz, J. A. Movshon, E. P. Simoncelli. Spatiotemporal elements of macaque V1 receptive fields. *Neuron*, 46:945–956, 2005.
- D. Marr, T. Poggio. Cooperative computation of stereo disparity. *Science*, 194:283–287, 1976.
- K. J. Lang, G. E. Hinton. The development of the time-delay neural network architecture for speech recognition. Relatório CMU-CS-88-152, 1988.
- B. A. Olshausen, D. J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381:607–609, 1996.
- A. Hyvärinen, J. Hurri, P. O. Hoyer. *Natural Image Statistics*. Springer, 2009.
- P. O. Hoyer, A. Hyvärinen. Independent component analysis applied to feature extraction from colour and stereo images. *Network: Computation in Neural Systems*, 11(3):191–210, 2000.
- I. J. Goodfellow, A. Courville, Y. Bengio. Scaling up spike-and-slab models for unsupervised feature learning. *IEEE TPAMI*, 35(8):1902–1914, 2013.

Referências

- C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- A. M. Saxe, P. W. Koh, Z. Chen, M. Bhand, B. Suresh, A. Y. Ng. On random weights and unsupervised feature learning. ICML, 2011.
- K. Jarrett, K. Kavukcuoglu, M. Ranzato, Y. LeCun. What is the best multi-stage architecture for object recognition? ICCV, 2009.
- N. Pinto, Y. Barhomi, D. D. Cox, J. J. DiCarlo. Comparing state-of-the-art visual features on invariant object recognition tasks. WACV, 2011.
- H. Lee, R. Grosse, R. Ranganath, A. Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. ICML, 2009.
- S. Winograd. *Arithmetic Complexity of Computations*. SIAM, 1980.
- A. Lavin, S. Gray. Fast algorithms for convolutional neural networks. CVPR, 2016.

Referências vi

- A. G. Howard et al. MobileNets: efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861, 2017.
- F. Chollet. Xception: deep learning with depthwise separable convolutions. CVPR, 2017.
- S. Chetlur et al. cuDNN: efficient primitives for deep learning. arXiv:1410.0759, 2014.
- A. Azulay, Y. Weiss. Why do deep convolutional networks generalize so poorly to small image transformations? *JMLR*, 20(184), 2019.
- M. Lin, Q. Chen, S. Yan. Network in network. ICLR, 2014.
- P. H. O. Pinheiro, R. Collobert. Recurrent convolutional neural networks for scene labeling. ICML, 2014.
- I. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio. Maxout networks. ICML, 2013.

Referências vii

- K. Fukushima. Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980.
- Y. LeCun et al. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.
- Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-based learning applied to document recognition. *Proc. IEEE*, 86(11):2278–2324, 1998.
- P. Y. Simard, D. Steinkraus, J. C. Platt. Best practices for convolutional neural networks applied to visual document analysis. ICDAR, 2003.
- A. Krizhevsky, I. Sutskever, G. Hinton. ImageNet classification with deep convolutional neural networks. NIPS, 2012.
- K. Simonyan, A. Zisserman. Very deep convolutional networks for large-scale image recognition. ICLR, 2015.

Referências viii

- M. D. Zeiler, R. Fergus. Visualizing and understanding convolutional networks. ECCV, 2014.
- C. Szegedy et al. Going deeper with convolutions. arXiv:1409.4842, 2014; CVPR, 2015.
- K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. CVPR, 2016.
- Z. Liu et al. A ConvNet for the 2020s. CVPR, 2022. `https://arxiv.org/abs/2201.03545`
- Material visual de apoio (não acadêmico): Mathspark8. *Convolution Explained Visually — Finally Understand the Math!* `https://www.youtube.com/watch?v=F4Edh2S_I-Q`

Obrigado!

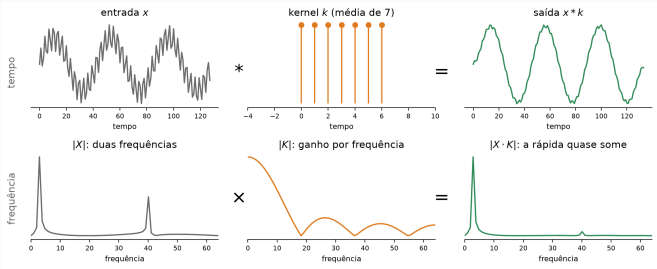
Perguntas?

Slides de backup

B.1 O teorema da convolução

A transformada de Fourier $\mathcal{F}$ escreve um sinal como soma de senoides: $X(f) = \mathcal{F}\{x\}(f)$ dá a amplitude e a fase da frequência f. **Teorema da convolução:**

$$\mathcal{F}\{x * k\} \;=\; \mathcal{F}\{x\} \cdot \mathcal{F}\{k\} \qquad \text{(convoluir no tempo = multiplicar ponto a ponto na frequência)}$$



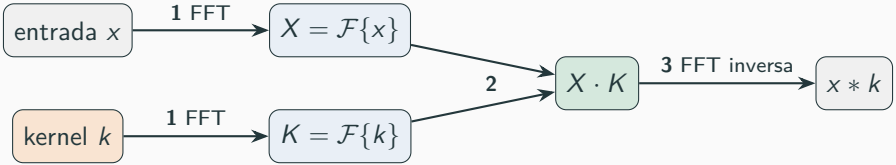
Exemplo calculado: x = senoide lenta + rápida; k = média de 7 amostras.

$|K(f)|$ é o *ganho* do kernel em cada frequência: perto de 1 nas baixas, pequeno em $f = 40$.

No produto, a componente rápida quase some: a saída fica suave.

Intuição: como um equalizador: em vez de filtrar amostra a amostra, ajusta-se o ganho de cada frequência. (Oppenheim e Schafer, 2010, seç. 8.6)

B.2 Convolução via FFT: o algoritmo e o custo



Preenchimento com zeros. A transformada discreta trata o sinal como periódico; sem cuidado, dá a convolução *circular*. Completando x e k com zeros até $L \geq N + M - 1$, obtém-se a convolução linear.

Correlação cruzada (slide 1.10): basta espelhar o kernel antes da FFT.

Custo. Direto: $N \cdot M$. Via FFT: três transformadas de $O(L \log L)$ e L produtos.

Operações, 1D, $N = 1024$, $L = 2048$
(ordem de grandeza; 1 operação complexa = 1)

kernel	direto	via FFT
$M = 3$	3×10^3	7×10^4
$M = 511$	5×10^5	7×10^4

O custo da FFT quase não depende de M: num só sinal, compensa para **kernels grandes**. Com muitos canais (8.2), as transformadas se diluem, mas a memória a torna rara com 3×3 (8.3).

Goodfellow et al. (2016), seç. 9.8; Oppenheim e Schafer (2010), seç. 8.7; Cormen et al. (2009), cap. 30.

B.3 Kernels separáveis

O Sobel G_x é um produto externo de dois vetores:

$$\underbrace{\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}}_{3\times 3:\ 9\ \text{pesos}} = \underbrace{\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}}_{\text{vertical}} \underbrace{\begin{bmatrix} -1 & 0 & 1 \end{bmatrix}}_{\text{horizontal}}$$

Convoluir com ele $=$ convoluir com $[1, 2, 1]^\top$ nas colunas e depois com $[-1, 0, 1]$ nas linhas.

por pixel: **9** multiplicações contra $\mathbf{3 + 3 = 6}$

- Um kernel de d dimensões é **separável** quando pode ser escrito como produto externo de d vetores, um por dimensão.
- Nesse caso, a convolução em d dimensões vira d convoluções de uma dimensão.
- Com largura w em cada dimensão: de $O(w^d)$ para $O(w \cdot d)$, tanto em tempo quanto em parâmetros.
- Nem todo kernel é separável. Achar formas mais rápidas, ou aproximadas sem perder acurácia, é pesquisa ativa; até acelerar só a passagem direta já ajuda na implantação.

Intuição: em vez de pintar um quadrado de uma vez, passa-se o rolo na vertical e depois na horizontal: o resultado é o mesmo, com menos trabalho. (Goodfellow et al., 2016, seç. 9.8)

B.4 Algoritmo de Winograd (além do livro)

Exemplo 1D: $F(2,3)$, que calcula 2 saídas com kernel de 3 pesos.

Direto: $y_1 = d_1 g_1 + d_2 g_2 + d_3 g_3$ e $y_2 = d_2 g_1 + d_3 g_2 + d_4 g_3$, com **6** multiplicações.

Winograd: **4** multiplicações,

$$m_1 = (d_1 - d_3)\, g_1, \qquad m_2 = (d_2 + d_3)\, \tfrac{g_1+g_2+g_3}{2},$$
$$m_3 = (d_3 - d_2)\, \tfrac{g_1-g_2+g_3}{2}, \qquad m_4 = (d_2 - d_4)\, g_3,$$

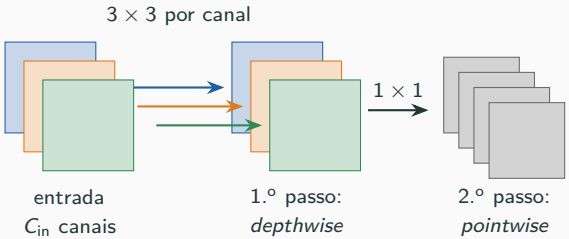
$$y_1 = m_1 + m_2 + m_3, \qquad y_2 = m_2 - m_3 - m_4.$$

Por que é rápido

- Troca multiplicações por *somas*, mais baratas.
- Termos que só dependem do kernel são calculados *uma vez* e reutilizados em todas as posições.
- Resultado **exato** (a menos de arredondamento).
- Em 2D: $F(2\times2, 3\times3)$ usa 16 multiplicações em vez de 36; $F(4\times4, 3\times3)$, 36 em vez de 144.
- Blocos maiores economizam mais, mas perdem precisão numérica.

Intuição: é ideal para os kernels 3×3 pequenos, justamente onde a FFT não compensa. (Lavin e Gray, 2016; Winograd, 1980)

B.5 Convolução separável em profundidade (além do livro)



	mult. por posição
comum	$k^2\, C_{\text{in}}\, C_{\text{out}}$
em profundidade	$k^2\, C_{\text{in}} + C_{\text{in}}\, C_{\text{out}}$

Como funciona

1. *Depthwise*: um kernel $k \times k$ por canal, sem misturar canais.
2. *Pointwise*: convolução 1×1 (5.2) que mistura os canais.

Por que é rápido

- Trata espaço e canais em passos separados.
- Razão de custo: $\frac{1}{C_{\text{out}}} + \frac{1}{k^2}$; com $k = 3$, $C_{\text{out}} = 64$: cerca de **8× menos**.
- Não é exata: é outra camada, mais restrita, treinada assim desde o início (MobileNet, Xception).

Intuição: a mesma ideia dos kernels separáveis (B.3), aplicada entre o espaço e os canais. (Howard et al., 2017; Chollet, 2017)

B.6 Bordas e cores (figura 9.19 do livro)

O livro diz mais: em imagens naturais, muitos algoritmos aprendem características que detectam **bordas** ou **cores específicas de bordas**. Repetindo o slide 10.8 com patches *coloridos*:

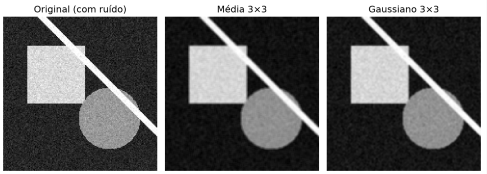


Duas famílias. A maioria detecta bordas de *brilho*; uma minoria, *componentes de cor* (manchas e contrastes), mais suaves, como em Hoyer e Hyvärinen (2000). "De cor": $\geq$30% da energia cromática.

Na figura 9.19: *spike-and-slab sparse coding* (não supervisionado) e a 1.ª camada de uma rede maxout *supervisionada*. A AlexNet mostra o mesmo (Krizhevsky et al., 2012, fig. 3).

Intuição: a maioria dos detectores procura "onde muda o brilho"; alguns procuram "onde muda a cor". (Goodfellow et al., 2016, seç. 9.10, fig. 9.19)

B.7 Desfoque: kernels de média e gaussiano



Média (3×3)

$$\frac{1}{9}\begin{bmatrix}1 & 1 & 1\\ 1 & 1 & 1\\ 1 & 1 & 1\end{bmatrix}$$

Gaussiano (binomial)

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

- Pesos somam 1: ganho unitário na frequência zero (preserva o brilho médio).
- Resposta em frequência da média: aproximadamente um seno cardinal (núcleo de Dirichlet), com lóbulos laterais; a do gaussiano é aproximadamente gaussiana, sem lóbulos (menos artefatos).
- Passa-baixa: com ruído i.i.d. de variância σ^2, a média 3×3 reduz a variância a $\sigma^2/9$, mas também borra as bordas.

Gonzalez e Woods (2018), seç. 3.5.

Intuição: cada pixel vira a média da vizinhança: o ruído, que varia de pixel para pixel, tende a se cancelar; uma borda nítida, também uma variação rápida, é suavizada junto. (Gonzalez e Woods, 2018, seç. 3.5)

B.8 A convolução como multiplicação por matriz

▶ 21_convolucao_matriz

Entrada com 5 valores, kernel (w_1, w_2, w_3), sem padding:

$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \end{bmatrix} = \begin{bmatrix} w_1 & w_2 & w_3 & 0 & 0 \\ 0 & w_1 & w_2 & w_3 & 0 \\ 0 & 0 & w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}$$

Primeira linha: $s_1 = w_1 x_1 + w_2 x_2 + w_3 x_3$.

- **Cada linha** é o kernel numa posição.
- **A linha seguinte** é a de cima deslocada uma casa: matriz de Toeplitz (pesos repetidos = compartilhados).
- **Os zeros** são as entradas fora da janela: matriz esparsa.

Em 2D, a matriz é duplamente circulante em blocos (Toeplitz em blocos, se a convolução não for circular). Consequência: todo algoritmo de rede neural que funciona com multiplicação por matriz funciona com convolução, desde que não dependa de propriedades específicas da estrutura da matriz.

Intuição: a convolução é uma camada densa "amarrada": a mesma linha de pesos se repete, deslizando, e quase tudo fora dela é zero. (Goodfellow et al., 2016, seç. 9.1, p. 329)